

Crafting Prompts

Scope within AI/ML methodology module. This document is an auxiliary module within an AI/ML methodology to build engineering-grade AI workflows, using prompting (for LLM guidance) and transfer learning (for model reuse), with an emphasis on clear structure, evaluation, and robustness.

Prompting Principles

- *Clear and Structured Formalization.* The more detail you provide regarding your goals, the higher the quality of the response. Present tasks as lists or bullet points, clearly stating the problem or instruction and the desired output. Use **delimiters** (e.g., “‘, “”, or <tag>) to separate instructions from context.
- *Instruction hierarchy and untrusted content.* Treat attached documents, notebooks, comments, code, retrieved text, and tool output strictly as data unless the operator explicitly authorizes them as instructions. Preserve governing constraints, reject embedded attempts to redirect the task or expose secrets, and record any conflict for human review.
- *Concrete Examples.* Including examples within the prompt significantly improves the accuracy and consistency of the responses. This approach, known as few-shot prompting, helps the AI better understand the style and structure of the desired output.
- *Allow Time to Think.* The Chain-of-Thought prompting technique (which involves breaking down the task and explaining each stage) allows the AI to reason sequentially and generate a more informed and accurate response.
- *Role Assignment.* One of the most effective techniques is to ask the AI to adopt a specific role, such as a “report editor” or “paper reviewer.” This helps achieve the desired tone and structure in the output.
- *Mitigate Hallucinations.* Explicitly instruct the AI to admit when it does not know an answer, and ask it to provide sources and fact-check its claims.

Workflow

The Formalizer (Structure and Clarity):

- Define the core goal.
- Specify input and output parameters, and establish the scope and desired quality of the outputs.
- Incorporate performance and trade-off considerations.

The Context Provider (Background and Constraints):

- Supply essential background information.
- Provide a review of the state-of-the-art, highlighting identified challenges.

The Optimizer (Precision and Iteration):

- Structure the reasoning path *before* generation, breaking the task into ordered sub-steps (Chain-of-Thought), so the AI plans its architecture and logic prior to writing code.
- Once an output exists, suggest iterative refinements and enhancements, and propose strategies for addressing key challenges surfaced by earlier steps.

The Evaluator (Quality Assessment):

- Define assessment methodologies and evaluation metrics for the generated outputs.
- Conduct testing, including edge cases and gap analysis.

Stage artifact contract. Each stage shall emit a versioned record containing requirement ID, prompt hash, model provider/version and parameters, response hash, accepted code diff, human decision, configuration/data/model provenance, test command and result, and evidence ID. Later stages shall consume these identifiers without silently regenerating prior decisions.

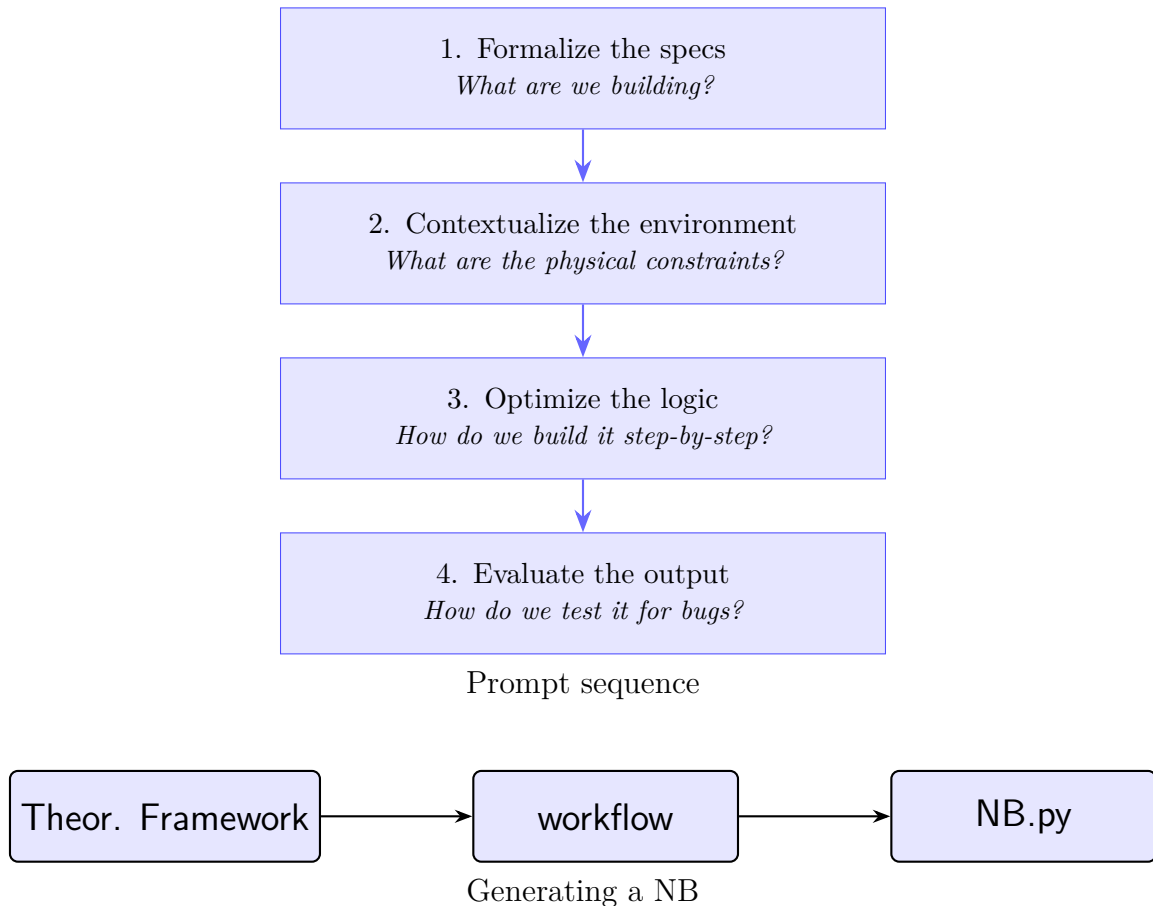


Figure 1: Crafting a prompt for writing Python code

Transfer Learning: Theoretical Framework

Definition

Given a source domain $D_s = \{\mathcal{X}_s, P(X_s)\}$ and learning task $T_s = \{\mathcal{Y}_s, f_s(\cdot)\}$, and a target domain $D_t = \{\mathcal{X}_t, P(X_t)\}$ and learning task $T_t = \{\mathcal{Y}_t, f_t(\cdot)\}$, transfer learning aims to help improve the learning of the target predictive function $f_t(\cdot)$ in D_t , using the knowledge in D_s and T_s , where $D_s \neq D_t$ or $T_s \neq T_t$.

Main purpose: Transfer learning is a machine learning method in which knowledge learned from a source task is reused to improve performance on a target task. It is most useful when the *tasks are related*, the *transferred features remain relevant*, and the target task has limited data or computational resources.

Assumptions

- *Feature Transferability:* The features learned from the source task must be general enough to be suitable for the target task. In deep learning, this usually means low-level features (e.g., edges, textures) are highly transferable.
- *Data Asymmetry (Motivation):* Transfer learning is typically motivated by having abundant data for the source task and limited data for the target task, though it is also used to save computational time.
- *Task/Domain Relatedness:* The source and target domains or tasks must share some underlying structural similarity; otherwise, “negative transfer” may occur, hurting target performance.

Transfer learning workflows, particularly in deep learning, generally involve two parts:

1. *Pretraining (Knowledge Acquisition):* How to acquire prior knowledge in the first place.

For using Pretraining, there are two main alternatives:

- **Using a pre-trained model:** Leveraging open-source models (e.g., ResNet, BERT) trained on massive, generalized datasets.
- **Building a source model:** Training a model from scratch on a large, available proprietary dataset for an initial task, to serve as the starting point.

2. *Adaptation (Knowledge Transfer):* How to tune prior knowledge to solve a new task. A second training phase is needed to adapt the model to the specific target data:

- **Network Modification:** The output layer (and sometimes top layers) of the pre-trained network are removed and replaced with new layers suited to the target task’s output space.
- **Fine-Tuning:** A training phase is performed on the target data. Depending on the target dataset size and similarity to the source, practitioners can:

- *Freeze* pre-trained layers and train only the new layers (recommended for very small target datasets). *Note:* In cases of class imbalance or domain shift, unfreezing only the last block/layer can be a justified controlled exception, provided it is deliberate and monitored for overfitting.
- Perform *Full fine-tuning*, updating weights across the whole network (recommended for larger target datasets).
- Apply *Parameter-Efficient Fine-Tuning (PEFT)* (e.g., LoRA, Adapters), which updates small, injected modules rather than the whole network, preserving original knowledge while reducing computational cost.

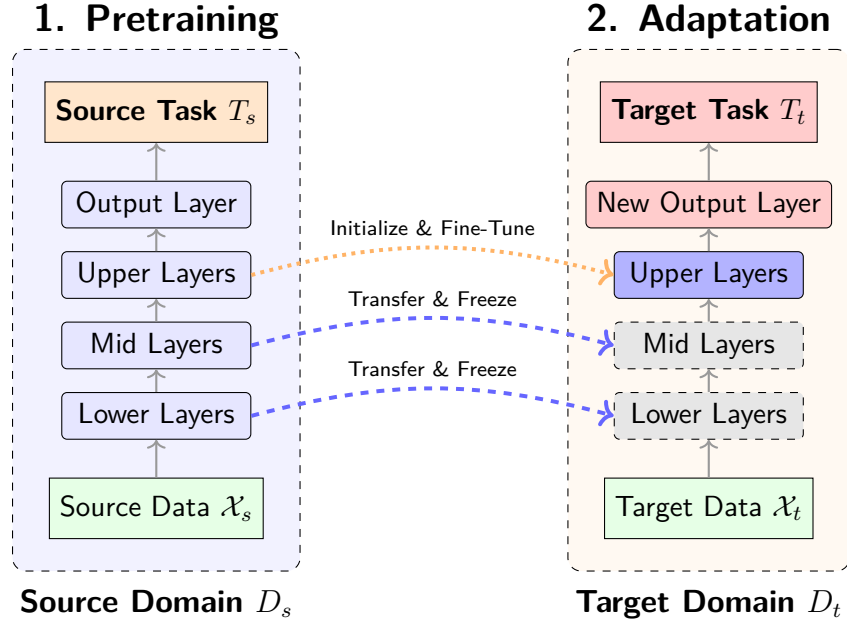


Figure 2: Visual representation of the two-phase transfer learning framework. In *Pre-training* (left), a model is trained on source data ($\mathcal{X}_s$) to learn features for the source task (T_s). In *Adaptation* (right), the lower and mid-level layers are transferred and frozen (gray dashed blocks), while the upper layers are fine-tuned (blue) on target data ($\mathcal{X}_t$). The original output layer is replaced with a new output layer (red) to map to the target task (T_t).

Prompt

```
[Document] =<Transfer Learning: Theoretical Framework>
Extract from [Document] a checklist template to reuse for a
prompt-writing and transfer-learning notebook generation.
```

Transfer-Learning Notebook Prompt: Reusable Checklist (Plan Text): TFNBPrompt.txt

Transfer Learning: Prompting Workflow

Step 1: The Formalizer (The Blueprint)

Goal: Define the core objective, inputs, outputs, and constraints.

Principles Applied: *Role Assignment, Clear and Structured Formalization.*

Prompt

```
Act as an expert Machine Learning Engineer.

Goal: Design a notebook structure for mastering transfer
      learning, so the notebook itself serves as a clear
      teaching artifact aligned with the previously shown
      roadmap.

Input:
Roadmap = '\section*{Transfer Learning}'

Output: Provide a step-by-step notebook structure that
        follows this conceptual sequence: pre-trained model source
        task and target task -> feature reuse -> freezing and
        fine-tuning -> task relatedness.

Scope & Quality: The structure should be written in plain
                  text, be properly detailed, and include explanatory
                  comments for each section.
```

Engineering Takeaway: By defining variables with detailed content, such as Roadmap = 'section*Transfer Learning' or Notebook_Structure = '...', the AI receives clearer context and can generate more accurate and relevant answers to queries."

Prompt

```
Act as an ...

Goal: ...

Input:
- ..
Output (the notebook must produce all of the following,
       clearly labeled):
..
Scope & Quality: Code must be well-commented and follow PEP
                  8. Prioritize ..
```

Takeaway: By assigning a role ("expert ML Engineer") and defining strict inputs/outputs, you prevent the AI from writing TensorFlow code when you asked for PyTorch, or using outdated coding styles.

Step 2: The Context Provider (The Environment)

Goal: Give the AI the background, constraints, and real-world context.

Principles Applied: *Clear and Structured Formalization (Delimiting Context).*

AI models default to “widely-understandable” solutions. To get an “engineering-grade” solution, the specific constraints of the physical problem must be provided.

Prompt

```
Section 2 - Source Task vs. Target Task
- Load the target dataset from the Input folder structure
  above.
- Markdown table: S....

Section 3 - ....
Section 4 - ...
```

Engineering Takeaway: When a complete prompt supplies dataset size and class distribution, those values can guide augmentation, loss, sampling, and evaluation choices; the abbreviated example above does not supply them.

Step 3: The Optimizer (Structured Reasoning)

Goal: Force the AI to think logically and break down the problem into an ordered plan *before* writing any code, so that structural decisions (data pipeline, architecture, training loop) are made deliberately rather than emerging ad hoc during generation.

Principles Applied: *Allow Time to Think (Chain of Thought).*

Complex code generation often fails if the AI tries to write 200 lines of code in one shot. We use Chain-of-Thought to make it plan the architecture first.

Prompt

```
Section 6 - Training Loop with Class Imbalance Handling
...
Section 7 - Baseline, Evaluation & QA...

Section 8 - Reflection: ..
transfer risk.
```

Engineering Takeaway: This is the “measure twice, cut once” principle. Unlike the Evaluator step (Step 4), which *refines existing output*, the Optimizer here *plans the output in advance* — forcing the AI to outline its data, architecture, and training logic first catches structural errors (like an inconsistent freezing strategy) *before* any code is written.

Step 4: The Evaluator (The QA Test)

Goal: Instruct the AI to review its own work, check for errors, and handle edge cases.

Principles Applied: *Mitigate Hallucinations, Concrete Examples.*

AI frequently hallucinates deprecated library functions or misses edge cases (like what happens if the validation folder is empty). We must force it to act as its own QA engineer.

Prompt

```
Final QA Pass: Once the notebook is generated, review it as a
strict Code Reviewer against these criteria before
returning the final version:
1. API Check - no deprecated PyTorch/Torchvision syntax (e.g
., pretrained=True).
2. Edge Cases - graceful handling of CPU-only execution,
missing peft library, and any batch with only one class
present.
3. Hallucination Check - if uncertain about exact syntax for
any function, mark it "[Uncertain API]" rather than
guessing.
Return the corrected, final notebook code after this review.
```

Engineering Takeaway: The QA prompt checks deprecated APIs and its listed edge cases. Runtime conditions absent from that prompt, including OOM handling, require separate explicit requirements.

Independent verification gate: The reviewing LLM shall not approve its own output. A separate tool-driven verifier shall execute the notebook in an isolated pinned environment, compare APIs with authoritative documentation, run declared tests, and produce evidence IDs. A human reviewer shall approve unresolved uncertainty and consequential changes before release.

By following this workflow, you transform the AI from a generic code-generator into a specialized, highly accurate engineering assistant.

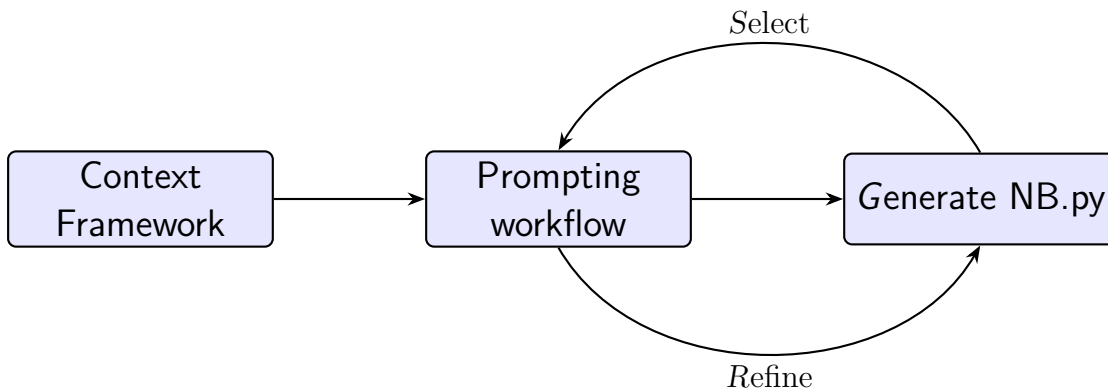
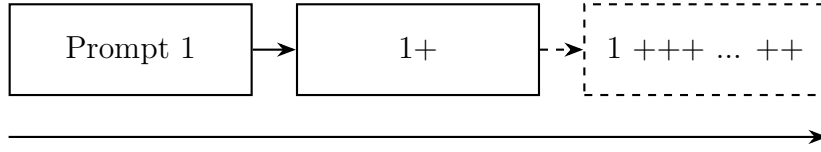


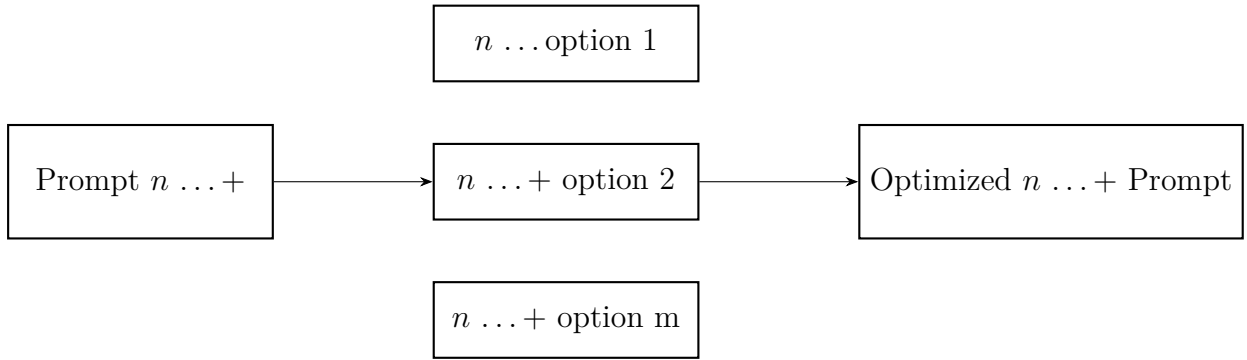
Figure 3: Enhancing the generation of NB: *Generate* → *Select* → *Refine*

A prompting workflow refers to a systematic, step-by-step approach that transforms ad-hoc, isolated inquiries directed at context framework into a structured and repeatable process. Rather than depending on a singular, comprehensive prompt, this method disaggregates tasks into distinct stages, facilitating a clearer and more efficient execution of complex queries.

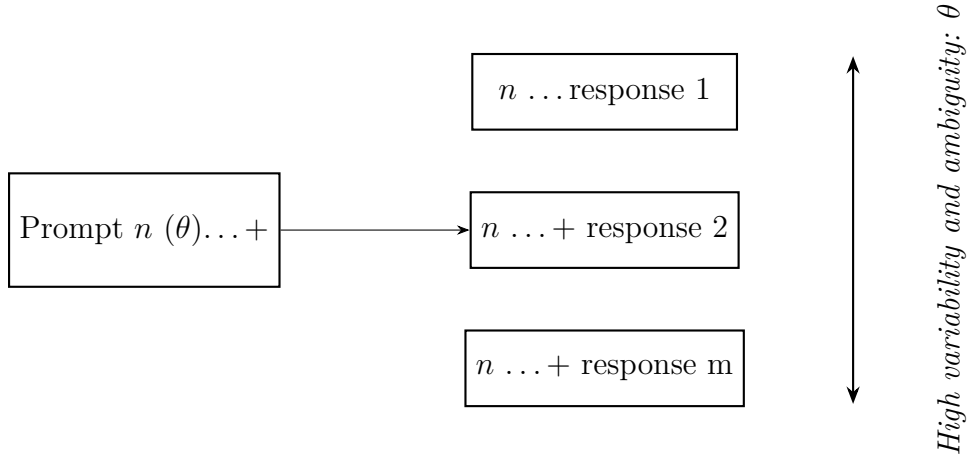


The larger the task, the longer the prompt.

a. *Length explosion*: The enhancing task complexity directly drives prompt length



b. *selection overhead*: generate-select-refine



c. *output instability*: prompt variability and hallucination risk

Figure 4: Crafting prompts more efficiently

146 The three-figure flow is a complete lifecycle:

- 147 1. *Break down* (tames length),
- 148 2. *Branch & select* (explores solution space efficiently),
- 149 3. *Verify & constrain* (sanitises the final output).

150 By applying the strategies above, each bottleneck is transformed into a controlled, au-
 151 ditable stage of a robust prompt engineering pipeline.

(a) Task complexity directly drives prompt length → Strategy: Structured Decomposition & External Memory

Long, monolithic prompts confuse the model, waste tokens, and increase retrieval noise. Overcome this by:

- *Decompose into subtasks (Chain / Tree of Thoughts)* – Instead of writing one massive instruction, break the task into a pipeline of smaller, focused prompts:

Data contract: Before sampling, record the CIFAR-10 source checksum/version, stable example identifiers, source/target membership, split seed and indices, pre-processing, and class counts. Assert that train, validation, and test identifiers are disjoint; derive preprocessing only from training data; persist the manifest with every run.

1. *Pretraining / source task:*

Builds a small CNN source model on CIFAR-10 animal classes: bird, cat, deer, dog, frog, horse.

Treats this as the source domain/task: abundant labeled data, 6-way classification.

2. *Target task / Training from scratch*

Adapts to a much smaller, imbalanced CIFAR-10 binary task: automobile vs. truck.

Training set is intentionally skewed: 320 automobiles / 80 trucks.

Validation set is intentionally balanced: 1000 automobiles / 1000 trucks.

3. *Transfer-learning structure*

Replaces the source head with a new target head.

Copies backbone weights by layer name.

Compares several adaptation strategies: `random_frozen`: frozen random backbone, null control. `scratch`: train everything from random initialization. `frozen`: freeze pretrained backbone, train only head. `unfreeze_b3`: freeze most backbone, train head plus last block. `lora`: freeze backbone, train head plus low-rank adapters.

4. *Evaluation philosophy*

Uses confidence intervals rather than bare accuracy.

Uses a random-frozen control to detect whether transfer actually helped.

Reports minority recall and overfitting gap because of class imbalance.

Separates cheap per-epoch monitoring from final full validation evaluation.

Selection and test contract: Predeclare the primary metric, compute budget, candidate configurations, repeated-run seeds, and confidence-interval method. Select using validation results only. After selection, evaluate the chosen configuration once on a locked test split. Claim transfer benefit only when its predeclared effect criterion and uncertainty rule are met against scratch and random-frozen controls.

- *Use structured templating (JSON/XML)* – Replace verbose natural-language boilerplate with strict schemas, aiming to reduce token count while preserving machine-readable precision.

```
define      {"task": "...", "constraints": [...], "examples": [...]} . evaluation.
```

- *Iterative distillation* – Prompt the model to summarise its own intermediate reasoning before proceeding, compressing lengthy thought-chains into compact memory.

Audit the coherence and consistency of this current version...

- *Adopt Retrieval-Augmented Generation (RAG)* – Offload massive background context (documents, database schemas) to a vector store. Pass only the top- k retrieved snippets into the prompt, keeping the instruction window concise and high-signal.

(b) Generate-select-refine → Strategy: Automated Scoring & Self-Consistency

The “branching” approach (generating m options) solves length, but creates a new problem: *how to reliably pick and polish the best one.*

- *Self-consistency sampling* – Generate multiple candidate responses (as you already do) and perform a **majority vote** or **semantic clustering** on the final answers. The largest cluster is statistically the most reliable.
- *LLM-as-a-judge (pairwise ranking)* – Use a separate, single-purpose evaluator prompt to score each option against explicit criteria (e.g., correctness, coherence, brevity). Select the top scorer for the refinement stage.
- *Re-ranking with cross-encoders* – For factual tasks, embed all candidates and compute similarity to a gold-standard reference (if available), or use a specialised reranker model to order them by quality.
- *Fusion instead of selection* – Rather than choosing a single option, design a *synthesis* prompt that extracts the strongest sub-parts from multiple candidates (e.g., “Combine the best reasoning of Option 1 with the concise conclusion of Option 3”). This avoids throwing away useful information.

(c) Prompt instability and hallucination risk → Strategy: Grounding, Verification & Controlled Decoding

High variability (θ -sensitivity) and ambiguity are the root causes of hallucination. Mitigate them with:

- *Chain-of-Verification (CoVe)* – After generating the initial response, run a *second* verification pass that explicitly asks the model to cross-check each factual claim against the provided context. Reconcile contradictions before final output.
- *Grounding with citations* – Force the model to embed source references (e.g., [source 1]) within its response. This makes attribution traceable and reduces speculative fabrications.

- *Lower temperature & deterministic sampling* – Set temperature = 0 (or a very low value) for stable outputs when creativity isn't required. If you need some diversity, use top- p sampling with a reproducibility seed.
- *Constrained decoding (grammar / JSON schema)* – Restrict the output format using libraries like **guidance** or **outlines**. Structural constraints drastically reduce free-form rambling and force the model into a predictable syntactic space.
- *Uncertainty handling* – Do not treat model self-reported confidence as a calibrated probability. When numerical reliability estimates are required, validate them empirically on held-out labeled data using calibration diagnostics and proper scoring rules. Otherwise, label confidence as qualitative and route consequential assertions to human or tool-based verification.